

When Does Prompting Help LLMs Write Formal Verification Specifications?

Weiqi

Supervisors: Prof. Lucas Cordeiro & Dr. Marie Farrell

April 21, 2026

The Problem: Passing $\neq$ Complete

Pass rate 94%

Recall 47%

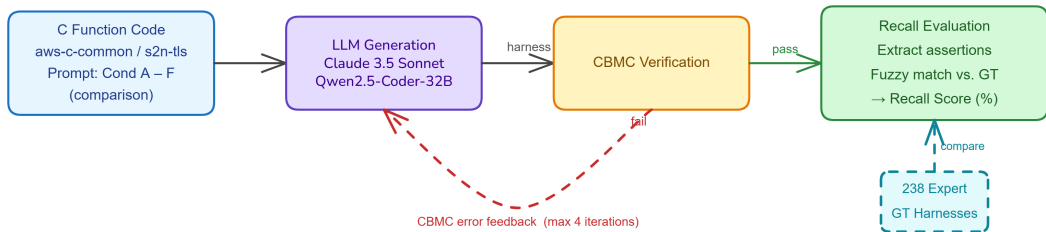
missing 53%

238 expert CBMC harnesses · aws-c-common +
s2n-tls

- CBMC only checks what you specify
- LLM harness compiles and passes ✓
- But **53% of GT properties missing**
— the ones that catch subtle bugs

**Passing CBMC $\neq$ Complete
Specification**

Experimental Pipeline



Research Questions

RQ1 — Baseline Completeness

How complete are LLM-generated verification harnesses,
and what types of properties are systematically missed?

RQ2 — Text Prompting

Does providing NL documentation or chain-of-thought reasoning improve postcondition recall?

RQ3 — Few-shot Example

Does a same-family GT harness example improve recall,
and what mechanisms drive the improvement?

RQ4 — Feedback Loop

Does iterative verifier error feedback improve recall?

Current study: CBMC (aws-c-common, s2n-tls) · Extension: ESBMC

How We Measure Recall

Mechanism:

- 1 Extract all `assert(...)` from GT and LLM harnesses via regex
- 2 Apply semantic normalization:
 - Equality direction: `a==b` $\leftrightarrow$ `b==a`
 - Pointer access: `list->len` $\leftrightarrow$ `list.len`
 - Old-state naming: `old_x` $\leftrightarrow$ `x_old`
- 3 $\text{Recall} = |\text{matched}| / |\text{GT}|$

Conservative lower bound — mismatches only under-count recall, never over-count.

Known limitations:

- `AWS_OP_SUCCESS` $\neq 0$ (macro not expanded)
 - `assert(a && b)` $\neq$ two separate asserts
- $\Rightarrow$ True recall $\geq$ measured recall

Planned validation:

- Manual spot-check: ~ 30 matched/unmatched pairs
- Taxonomy annotation: blind inter-rater + Cohen's κ

Why the Gap: CBMC Idioms

LLM harness — passes, but incomplete:

```
void aws_byte_buf_init_harness() {
    size_t capacity = nondet_size_t();
    __CPROVER_assume(capacity > 0);
    struct aws_byte_buf buf;
    aws_byte_buf_init(&buf, alloc, capacity);

    assert(buf.capacity == capacity);
    assert(buf.len == 0);
}
```

Ground truth — 4 missing assertions:

```
// Validity predicate:
assert(aws_byte_buf_is_valid(&buf));

// Frame condition:
assert(buf.allocator == alloc);

// Struct invariant:
assert(buf.len <= buf.capacity);

// Pointer coherence:
assert(buf.buffer != NULL
       || buf.capacity == 0);
```

Category	%	Why missed
VALIDITY_PRED	20%	predicate name unknown
LEN_INVARIANT	19%	struct contract unknown
FRAME_COND	17%	LLM focuses on changes
Top 3	56%	Tool-specific idiom gap — not misunderstanding the function

Text Cannot Teach Idioms — Examples Can

Text Prompting: No Effect

Condition	Claude	Qwen
A: baseline	46%	39%
B: + NL docs	47%	40%
C: + CoT	47%	35%
D: + NL + CoT	45%	38%
p vs A	> 0.28	

Ceiling $\sim 47\%$ — tool-specific idioms, not fixable by text

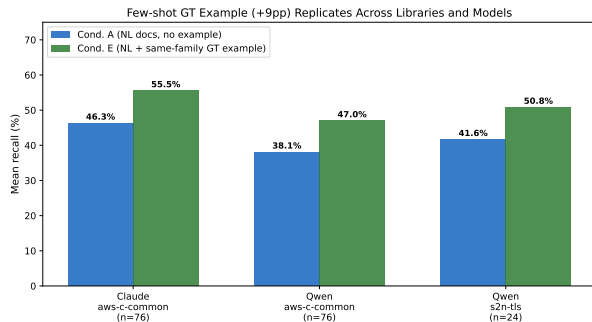
p : Wilcoxon signed-rank test vs. baseline (Cond. A)

One Same-Family Example: +9pp

Setting	A	E (+ example)
Claude / aws	46%	56% ($p=0.002$)
Qwen / aws	39%	47% ($p=0.037$)
Qwen / s2n	42%	51% ($p=0.003$)

Free · No retraining · Replicates

Replicated Across Models and Libraries



Setting	Δ	p
Claude / aws	+9.2pp	0.002
Qwen / aws	+8.9pp	0.037
Qwen / s2n	+9.5pp	0.003

- 2 models, 2 libraries
- Effect size $r = 0.6$ (large)
- Not a fluke

Recommendation

Always include one same-family
GT harness in the prompt.

Free · Replicable · +9pp recall

Open Questions:

- How many examples? (1 vs. 2–3)
- Generate example when no GT exists?
- Feedback paradox: recall flat as pass rises
- Generalise to ESBMC + Simulink?

Two mechanisms (ablation F):

- (1) **Generic idioms**: any example $\rightarrow$ +5.8pp
- (2) **Predicate names**: same family $\rightarrow$ +15pp